九阶 · Linux 与 Python 自学资料

17

九阶 · 关卡总表

125 关 · 按阶段推进与验收

学习指挥表

AYMLiu-Pzw · 2026-10-22

第 17 册

九阶 · 关卡总表

125 关 · 按阶段推进与验收

资料类型	学习指挥表
适用读者	需要统揽 125 关进度与验收标准时使用
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 阶段 0 · 环境与工具（7 关） ｜ learning-journey（日志仓库）
2. 阶段 1 · Linux 系统（45 关） ｜ 能力型，无仓库（判定靠阶段总验收 1-45）
3. 阶段 2 · Shell 脚本（12 关） ｜ shell-scripts（三个能讲出故事的脚本）
4. 阶段 3 · Python 脚本（18 关） ｜ python-scripts（重命名 / 天气 / 巡检 v0）
5. 阶段 4 · Docker（11 关） ｜ docker-scripts（把自己的脚本容器化）
6. 阶段 5 · 监控（6 关） ｜ monitor-demo（Prometheus + Grafana + 阈值告警）
7. 阶段 6 · Kubernetes（8 关） ｜ k8s-demo（含「Pod 起不来」踩坑记录）
8. 阶段 7 · 综合项目（8 关） ｜ server-health-check（王牌项目 v1）
9. 阶段 8 · AI / GPU / 推理部署（10 关） ｜ ai-inference-notes（部署笔记 + 踩坑记录）
10. 总索引 · 要查什么 → 翻哪册
11. 附录 A · 检验点（按关卡归位）
12. 附录 B · 外部资源清单（在哪学、只做哪一部分）

九阶 · 关卡总表

九阶 · 关卡总表

用法：**这是你的指挥中心**——做完一关就在对应正文核对完成判定；找不到下一步时，看这张表。**本表只摘录与索引；同名正文的完成判定 / 阶段验收卡是执行标准，若摘要不够就回正文核对。**

阶段 0 · 环境与工具（7 关） | learning-journey（日志仓库）

入场：无——从零开始（一台上网设备） | 毕业：learning-journey 仓库（日志 + README）

编号	关卡名	在哪学（教材位置）	完成判定
0-01	注册 GitHub 与建日志仓库	learning-journey 仓库 · README（建仓库与首次提交的记录）	仓库在线，第一次 commit 有记录
0-02	Git 最小闭环：clone / add / commit / push	16 册 · 第 2 页「日常循环四条命令；首次建仓七条 Git 子命令」（对应前置关 0-02）	不看笔记能独立推一次
0-03	装 Ubuntu 虚拟机并跑通第一条命令	平板路径：自行按 Termux 官方文档或搜索安装教程；电脑装法见 01 册 · 附录 E（WSL 为主 / VMware 为辅）	能进系统、能上网、能敲 ls；能说出你用的是哪一种环境（WSL / VMware / 平板）
0-04	编辑器：VS Code / code-server	VS Code 官网下载安装；平板路径：自行按官方文档或搜索搭建教程	能改文件并保存
0-05	平板环境：Termux 敲命令	平板路径：自行按 Termux 官方文档或搜索安装教程（装好即能敲命令）	平板上敲出第一条命令
0-06	日志模板与习惯	learning-journey 仓库 · 每日日志（写法参考 16 册 · 第 2 页「最小工作流」 + 第 3 页「提交规范」）	连续 3 天写日志（做了什么 / 卡在哪 / 明天第一件事）
0-07	阶段 0 · 学习总结练习：把日志仓库整理成别人能看懂的样子	16 册 · 书末「阶段 0 · 学习总结练习」	陌生人打开仓库能说出你在学什么；连续 3 天有提交记录；日志里至少 3 条"卡点 → 解决"

阶段 0 · 学习总结练习（见 16 册《Git 与 GitHub 速查》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。

阶段 1 · Linux 系统（45 关） | 能力型，无仓库（判定靠阶段总验收 1-45）

入场：0-07 完成 | 毕业：能独立处理"磁盘满 / 服务起不来"两类故障（1-45 验收）

编号	关卡名	在哪学（教材位置）	完成判定
1-01	认识终端与"命令+选项+参数"	01 册 · 第 0 章「开篇：终端与命令」→ 章首的关卡框	能说出提示符每一部分的含义；说出三件套各是什么；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）
1-02	用 --help 和 man 自救	01 册 · 第 0 章「开篇：终端与命令」→ 章首的关卡框	不看教程，用 --help 查出 ls 三个选项的含义
1-03	一切皆文件：读懂目录树	01 册 · 第 1 章「目录树与路径」→ 章首的关卡框	能画出 / 下的三大目录并说出用途
1-04	绝对路径 vs 相对路径	01 册 · 第 1 章「目录树与路径」→ 章首的关卡框	用两种写法各走一次：从任意位置到 /var/log 再回家目录
1-05	建与删：touch / mkdir -p / rm	01 册 · 第 2 章「文件与目录操作」→ 章首的关卡框	建出 a/b/c 三级目录再拆掉 · 能独立做出 02-★1-1（建多级目录） · 能独立做出 02-★1-3（批量创建）
1-06	复制与移动：cp 和 mv 的边界	01 册 · 第 2 章「文件与目录操作」→ 章首的关卡框	说清 cp 的“复制目录规则”和“目标路径解析规则”，并说清 mv 的移动/改名双功能 · 能独立做出 02-★1-4（cp 递归） · 能独立做出 02-★1-6（mv 双功能）

编号	关卡名	在哪学（教材位置）	完成判定
1-07	安全删除习惯 先学：1-14 ~ 1-16	01 册 · 第 5 章「用户与权限」→ 1-7 回验	学完第 5 章后再回验，不计阶段 1 首次判定。说清"删除靠的是目录写权限"，并演示一次删除前确认 · 能独立做出 02-★1-7（删除权限原理）
1-08	看文件：cat / less / head / tail / wc	01 册 · 第 3 章「查看与搜索」→ 章首的关卡框	用 head 和 tail 各取 5 行；在 less 里搜索并退出 · 能独立做出 02-★2-3（头尾查看）
1-09	搜内容与搜文件：grep / find	01 册 · 第 3 章「查看与搜索」→ 章首的关卡框	在日志里搜出指定字符串；找出今天新增的文件 · 能独立做出 02-★2-5（grep 过滤） · 能独立做出 02-★5-1（按名查找）
1-10	比较差异：diff	01 册 · 第 3 章「查看与搜索」→ 章首的关卡框	说出两个文件差在哪
1-11	重定向：> 覆盖与 >> 追加	01 册 · 第 4 章「管道与重定向」→ 章首的关卡框	演示两种写法的区别；说清为什么别重定向到正在写的文件 · 能独立做出 02-★2-1（创建与重定向）
1-12	管道与排序取前 N：sort / uniq	01 册 · 第 4 章「管道与重定向」→ 章首的关卡框	一条命令查出出现次数最多的前三项 · 能独立做出 02-★2-7（管道实战） · 能独立做出 02-★5-4（排序去重）
1-13	awk 入门：取列	01 册 · 第 4 章「管道与重定向」→ 章首的关卡框	取出日志第 2 列并统计
1-14	多用户模型：whoami / id / su / sudo	01 册 · 第 5 章「用户与权限」→ 章首的关卡框	说清 su 与 sudo 的区别；切一次身份并安全退出 · 能独立做出 02-★3-3（建用户）
1-15	rwx 与 chmod 符号法	01 册 · 第 5 章「用户与权限」→ 章首的关卡框	给脚本设成"自己读写执行、别人只读" · 能独立做出 02-★3-1（权限位解读） · 能独立做出 02-★3-4（越权验证） · 能独立做出 02-★3-6（共享组最小验收）
1-16	数字法与危险组合：644 / 755 / 777	01 册 · 第 5 章「用户与权限」→ 章首的关卡框	一次改对 644 与 755；说清 777 为什么危险 · 能独立做出 02-★3-5（chown 影响） · 能独立做出 02-★3-7（账号文件） · 能独立做出 02-★3-8（权限判断流程图）
1-17	看进程：ps aux 与 pgrep	01 册 · 第 6 章「进程管理」→ 章首的关卡框	找出某个进程的 PID 和它属于谁 · 能独立做出 02-★4-1（进程定位）
1-18	杀进程：信号、优雅 vs 强杀	01 册 · 第 6 章「进程管理」→ 章首的关卡框	先 TERM 后 KILL 处理一个卡死进程，并说清区别 · 能独立做出 02-★4-3（优雅 vs 强杀）
1-19	前后台与端口占用：jobs / nohup / fuser	01 册 · 第 6 章「进程管理」→ 章首的关卡框	让命令在后台跑；找出占用某端口的进程
1-20	看懂 df 与 du 的数字	01 册 · 第 7 章「磁盘与文件系统」→ 章首的关卡框	说出 df -h 每一列的含义；说清 df 与 du 的分工 · 能独立做出 02-★4-5（磁盘视角）
1-21	磁盘满了，第一眼看哪里	01 册 · 第 7 章「磁盘与文件系统」→ 章首的关卡框	不看教材，2 分钟内说出最大目录是哪个、多大
1-22	大文件与挂载	01 册 · 第 7 章「磁盘与文件系统」→ 章首的关卡框	造一个测试大文件再清理；能说出 umount 要先确认没有进程占用（target is busy 是怎么来的） · 能独立做出 02-★5-3（查找+执行）
1-23	查自己：ip / ip route / DNS	01 册 · 第 8 章「网络命令」→ 章首的关卡框	说出本机 IP、网关、DNS 各自怎么看
1-24	连通性：ping / curl / nc	01 册 · 第 8 章「网络命令」→ 章首的关卡框	说清"ping 不通"与"端口不通"的差别
1-25	端口与监听：ss -tulnp	01 册 · 第 8 章「网络命令」→ 章首的关卡框	找出某个端口是谁在听
1-26	四层排查法综合	01 册 · 第 8 章「网络命令」→ 章首的关卡框	面对"服务连不上"，按主机→端口→进程→日志走一遍
1-27	apt 包管理：装 / 卸 / 搜 / 锁报错	01 册 · 第 9 章「apt 包管理」→ 章首的关卡框	装一个工具再卸掉；说清 update 与 upgrade 的区别 · 能独立做出 02-★4-7（包管理）
1-28	服务状态：systemctl status / is-active	01 册 · 第 10 章「systemctl 与服务」→ 章首的关卡框	说出 loaded / active 各表示什么

编号	关卡名	在哪学（教材位置）	完成判定
1-29	起停与开机自启：start / stop / enable	01 册 · 第 10 章「systemctl 与服务」→ 章首的关卡框	停掉一个服务再恢复；说清 enable 与 start 的区别 · 能独立做出 02-★4-6（服务管理）
1-30	看服务日志：journalctl -u	01 册 · 第 10 章「systemctl 与服务」→ 章首的关卡框	从服务日志里找出最近一次失败的原因
1-31	环境变量：PATH 与 export	01 册 · 第 11 章「环境变量」→ 章首的关卡框	说清"命令为什么找不到"并修好一次
1-32	归档与压缩：tar / gzip / zip	01 册 · 第 12 章「归档与压缩」→ 章首的关卡框	打包一个目录再解开验证 · 能独立做出 02-★5-6（打包压缩）
1-33	第一个脚本：shebang、执行权限、退出码	01 册 · 第 13 章「Shell 脚本基础」→ 章首的关卡框	写出一个能跑并返回退出码的脚本
1-34	变量、if、for	01 册 · 第 13 章「Shell 脚本基础」→ 章首的关卡框	写一个"目录不存在就创建"的脚本
1-35	crontab 定时任务与绝对路径的坑	01 册 · 第 13 章「Shell 脚本基础」→ 章首的关卡框	让脚本每分钟跑一次并看到日志
1-36	阶段 1 综合：服务连不上，全流程自己走一遍	01·跨章	不看教材走完主机→端口→进程→日志，并说清每一步为什么 · 能独立做出 02-★2-6（递归搜索） · 能独立做出 02-★5-8（引号原理）
1-37	场景 1（上）：复现"推理服务起不来"	03 册 · 场景 1｜本地推理服务起不来（日志排查）→ 题目区（答案在书末）	在沙盒里把故障复现出来，并写下现象
1-38	场景 1（中）：从日志读到根因	03 册 · 场景 1｜本地推理服务起不来（日志排查）→ 题目区（答案在书末）	说出根因，并能指出日志里哪一行证明了它
1-39	场景 1（下）：修复 + 一页处理记录	03 册 · 场景 1｜本地推理服务起不来（日志排查）→ 题目区（答案在书末）	服务起来；记录按"现象→线索→命令→验证"写完
1-40	场景 3（上）：复现"服务连不上"	03 册 · 场景 3｜服务连不上（网络链路排查）→ 题目区（答案在书末）	复现故障，列出你要先看的三件事
1-41	场景 3（中）：四层链路定位	03 册 · 场景 3｜服务连不上（网络链路排查）→ 题目区（答案在书末）	按主机→端口→进程→日志定位到根因
1-42	场景 3（下）：修复 + 记录	03 册 · 场景 3｜服务连不上（网络链路排查）→ 题目区（答案在书末）	恢复服务并写完记录
1-43	场景 4（上）：进程与资源占用排查	03 册 · 场景 4｜服务与进程运维（含 GPU 进程定位）→ 题目区（答案在书末）	找出占资源最多的进程并说清依据
1-44	场景 4（中）：定位与安全处理	03 册 · 场景 4｜服务与进程运维（含 GPU 进程定位）→ 题目区（答案在书末）	定位根因并安全处理
1-45	阶段 1 · 学习总结练习：工单「磁盘满了 + 服务起不来」	01 + 03 · 综合	20 分钟内定位磁盘占用的大头；服务恢复并说清根因；有一份别人能看懂的根因分析（含恢复/回滚步骤）

阶段 1 · 学习总结练习（见 01 册《Linux 入门》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。

阶段 2 • Shell 脚本（12 关）　｜　shell-scripts（三个能讲出故事的脚本）

入场：1-35（crontab 定时）完成　｜　毕业：shell-scripts 三脚本 + crontab + systemd；本阶段不需要 Python

编号	关卡名	在哪学（教材位置）	完成判定
2-01	五件套地图：先看懂 5 个脚本各解决什么问题 先学：1-35	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	能用一句话说出每个脚本解决什么问题
2-02	backup.sh：备份 + 日期后缀	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	脚本能生成带日期的备份文件
2-03	backup.sh 排错：路径空格与变量引用	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	故意造一个带空格的文件名，脚本仍能处理
2-04	disk_alert.sh：取数 + 阈值判断	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	磁盘超 80% 时输出告警
2-05	接进 crontab：绝对路径的坑	01 册 · 第 13 章「Shell 脚本基础」→ 章首的关卡框	定时任务真的跑了，日志按时新增
2-06	service_guard.sh：服务挂了自动拉起	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	手动停服务后，脚本能把它拉起来
2-07	把脚本变成 systemd 服务	03 册 · 场景 5	脚本收尾（衔接 Shell 脚本任务）→ 题目区（答案在书末）	systemctl status 能看到自己的服务
2-08	加日志与退出码：让别人看得懂	01 册 · 第 13 章「Shell 脚本基础」→ 章首的关卡框	出错时退出码非 0，日志里能看出哪一步挂的
2-09	参数与用法：$1 / usage / 退出码约定	03 册 · 场景 5	脚本收尾 → 参数与用法示范（$1 / case）	脚本有 --help、能收参数、参数错时给用法
2-10	整理成仓库：shell-scripts + README	产出到 GitHub 仓库 shell-scripts；README 写法见 16 册《一个合格的 README 长什么样》	脚本都在仓库里，README 写清用法
2-11	三分钟讲述：这三个脚本解决什么问题	参考 10 册 · 7-07「三分钟讲述稿与 10 分钟深挖」	不看书讲清"为什么做、怎么做、踩了什么坑"
2-12	阶段 2 · 学习总结练习：5 件套脚本 + crontab + systemd	03 · 场景5 + 产出	陌生人照 README 能跑通三个脚本；定时任务真的执行过（日志为证）；手动停服务后能被拉起；每个脚本能讲 3 分钟

阶段 2 · 学习总结练习（见 03 册《Linux 练习册 · 进阶篇》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。
学到这 → 三个脚本都能讲故事了（全程不需要 Python）；下一步：先学 04 册第 0~4 章，再做 3-00（05 册阶段 1~4 的 20 道 ★题），最后进阶段 3。

阶段 3 • Python 脚本（18 关）　|　python-scripts（重命名 / 天气 / 巡检 v0）

入场：**先读 04 册第 0～4 章**，再做 3-00（05 册阶段 1～4 的 20 道 ★ 题）；04 第 0～4 章未学也没有把握时，不得跳过；有基础者可用这次诊断快过，低于 18 道按题号回 04 补。命令功底来自阶段 1　|　毕业：python-scripts（重命名 / 天气 / 巡检 v0）

阶段 2 与阶段 3，各管什么
Shell（阶段 2）：让机器自己干活——文件、进程、定时任务；全程不需要 Python。
Python（阶段 3，从 3-00 起）：处理数据、调接口——文件读写 / JSON / requests / 日志 / subprocess。
两条线在哪合流：4-05 把脚本打成镜像 → 5-06 用 Python 拉指标 → 7-02 长成王牌项目。

编号	关卡名	在哪学（教材位置）	完成判定
3-00	Python 基础学后诊断（先读 04 第 0～4 章）	05 册 · 阶段 1~4 的题目（题号 1-1 ～ 4-8；答案在书末）	20 道基础题一次做对 18 道以上；做错的回 04 对应章节补，不重学整章 · 能独立做出 05-★1-1（print 多参数） · 能独立做出 05-★1-3（变量与重赋值） · 能独立做出 05-★1-5（字符串操作） · 能独立做出 05-★1-7（数字运算） · 能独立做出 05-★1-8（变量命名） · 能独立做出 05-★2-1（if/else） · 能独立做出 05-★2-3（for + range） · 能独立做出 05-★2-5（while 计数器） · 能独立做出 05-★2-6（while + break） · 能独立做出 05-★2-8（嵌套循环） · 能独立做出 05-★3-1（list 基本操作） · 能独立做出 05-★3-3（dict 增删改查） · 能独立做出 05-★3-5（tuple/set） · 能独立做出 05-★3-6（list + tuple） · 能独立做出 05-★3-8（通讯录） · 能独立做出 05-★4-1（def 定义与调用） · 能独立做出 05-★4-3（return 与多返回值） · 能独立做出 05-★4-5（import os） · 能独立做出 05-★4-7（重构） · 能独立做出 05-★4-8（print vs return）
3-01	模块与导入：import 与 os 基本用法	04 册 · 第 5 章「模块与导入：import、os、time、random」→ 章首的关卡框	写出"列出某个目录下所有文件"的脚本；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）
3-02	路径与时间：os.path 与 time	04 册 · 第 5 章「模块与导入：import、os、time、random」→ 章首的关卡框	脚本打印"今天日期 + 当前工作目录"
3-03	读写文件：open / with	04 册 · 第 6 章「文件操作：open/with、json」→ 章首的关卡框	读写一个 txt，并说清 with 帮你做了什么 · 能独立做出 05-★5-1（open 读文件） · 能独立做出 05-★5-7（读文件统计）
3-04	JSON：load 与 dump	04 册 · 第 6 章「文件操作：open/with、json」→ 章首的关卡框	把字典存成 json 再读回来 · 能独立做出 05-★5-3（json 读写） · 能独立做出 05-★5-8（w vs a）
3-05	异常处理：try / except / finally	04 册 · 第 7 章「异常处理：try/except/finally」→ 章首的关卡框	文件不存在时给人话提示，不甩一堆报错 · 能独立做出 05-★5-5（try/except）
3-06	网络请求：requests 基础	04 册 · 第 8 章「网络请求：requests」→ 章首的关卡框	拿到接口返回并打印状态码

编号	关卡名	在哪学（教材位置）	完成判定
3-07	网络异常：超时 / 非 200 / 格式不对	04 册 · 第 8 章「网络请求：requests」→ 章首的关卡框	断网和乱数据都能给出明确提示，报错不乱
3-08	命令行参数：argparse	04 册 · 第 9 章「命令行参数：argparse」→ 章首的关卡框	python3 weather.py 杭州 能跑 · 能独立做出 05-★6-3（argparse 参数） · 能独立做出 05-★6-4（argparse 默认值）
3-09	日志：logging 替掉 print	04 册 · 第 10 章「日志：logging」→ 章首的关卡框	日志带时间戳、能分级 · 能独立做出 05-★6-5（logging 基本）
3-10	调用系统命令：subprocess	04 册 · 第 11 章「进程调用：subprocess」→ 章首的关卡框	脚本执行 df -h 并取出使用率 · 能独立做出 05-★6-7（subprocess 基本） · 能独立做出 05-★6-8（三模块综合）
3-11	场景 1（上）：批量重命名——先想清楚再动手	06 册 · 场景 1 | 批量文件重命名工具 → 题目（答案在书末）	能写出改名计划（哪些文件、改成什么）
3-12	场景 1（下）：实现 + 冲突处理 + 验证	06 册 · 场景 1 | 批量文件重命名工具 → 题目（答案在书末）	测试目录里的文件全部改名成功、无覆盖（附录 A 给的 10 个示例文件即可；想加量就用循环生成——判定看的是"一个不落、一个不覆盖"）
3-13	场景 2：天气查询器（API + 参数 + 异常）	06 册 · 场景 2 | 天气查询器 → 题目（答案在书末）	换城市能查、断网时有提示、结果能存文件
3-14	场景 3（上）：本机巡检 v0——三个检查函数	06 册 · 场景 3 | 本机巡检工具雏形（王牌项目 v0） → 题目（答案在书末）	能输出磁盘 / 内存 / 负载三项
3-15	场景 3（下）：报告 + 日志 + 生成 v0 产物	06 册 · 场景 3 | 本机巡检工具雏形（王牌项目 v0） → 题目（答案在书末）	在当前实验目录一次运行生成报告，日志有开始/完成；README、打包、clone 留到后续阶段，不作为本关验收。
3-16	场景 4：Nginx 日志分析器（样例日志）	06 册 · 场景 4 | Nginx 日志分析器 → 题目（答案在书末）	统计出访问量前十的 IP
3-17	阶段 3 · 学习总结练习：本机巡检 v0（磁盘 / 内存 / 负载）	06 册 · 书末“阶段 3 · 学习总结练习”	在 Ubuntu 实验目录运行 `python3 server_check.py`，生成三项结果和报告；当前环境触发阈值时显示告警；日志有本次开始/完成；能讲清三个检查函数各在查什么。只收 v0；argparse/命令失败处理见 7-02，README 见 7-06。

阶段 3 · 学习总结练习（见 06 册《Python 练习册 · 进阶篇》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。
学到这 → 可以开始 Docker（阶段 4）；巡检 v0 留着，阶段 7 要用。

阶段 4 • Docker（11 关） | docker-scripts（把自己的脚本容器化）

入场：3-17 完成 | 毕业：docker-scripts（把自己的脚本容器化）

编号	关卡名	在哪学（教材位置）	完成判定
4-00	YAML 是什么	07 册《Docker入门》· 关卡 4-00（该关即一节，标题同名）	能说出上面那段里哪些是键、哪些是值；能指出 "8080:80" 属于哪个键；能说出为什么不能用 Tab 缩进；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）
4-01	镜像与容器是什么：跑起第一个容器	07 册《Docker入门》· 关卡 4-01（该关即一节，标题同名）	能用自己的话说清镜像和容器的区别；docker run hello-world 能跑出输出；能说出 docker images 和 docker ps -a 各看的是什么；能解释 STATUS 里的 Exited (0) 是什么意思 回滚方式：实验完把容器清掉即可——docker rm happy_allen（把 NAMES 换成你自己那行）；镜像想删就 docker rmi hello-world。删了都不影响系统；能成功拉到镜像（先按 07 册「开始之前」配好镜像加速器）

编号	关卡名	在哪学（教材位置）	完成判定
4-02	六件套：pull / run / ps / logs / exec / stop·rm	07 册《Docker入门》· 4-02	六节命令能不看笔记敲一遍；能解释 127.0.0.1:8080:80 只监听回环地址；端口改 127.0.0.1:18080 后先预测地址与 docker port 输出；错误标签从 docker run / pull 的拉取错误判定，不用空容器 / 空日志猜根因；能区分 stop 后端口释放与 rm 后容器名/磁盘清理。回滚：stop + rm，镜像按需 rmi
4-03	数据卷与端口映射	07 册《Docker入门》· 关卡 4-03（该关即一节，标题同名）	能说出"为什么容器删了数据就没了"，并演示一次数据卷保住数据；能自己起一个 nginx 并用 curl 验证 200；能说出 -p 8080:80 里前后两个数字各是什么；能说出什么该挂卷、什么不用 回滚方式：实验容器 docker rm -f 掉；挂载目录在 ~/lab 里，删掉不影响系统
4-04	写第一个 Dockerfile	07 册《Docker入门》· 关卡 4-04（该关即一节，标题同名）	能不看笔记写出 FROM / RUN / COPY / CMD 四行；能说清 RUN 和 CMD 的区别（构建时 vs 启动时）；能说清 build 命令最后那个点是什么；能说出 COPY 为什么要放在后面 回滚方式：练废的镜像 docker rmi my-hello:1.0 删掉重来；~/lab/myimage 里的文件随时可改
4-05	把自己的脚本打包成镜像跑起来	07 册《Docker入门》· 关卡 4-05（该关即一节，标题同名）	自己的脚本能在容器里跑出结果；能说出 WORKDIR 解决了什么问题；改一处脚本后重新 build，能说清"哪几层用了缓存"；用 -e 把阈值传进去并触发一次告警 回滚方式：docker rmi disk-alert:1.0 删镜像；脚本原件在 ~/lab 和 shell-scripts 仓库里，没被改动过
4-06	镜像瘦身与分层	07 册《Docker入门》· 关卡 4-06（该关即一节，标题同名）	能用 docker history 说出哪一层最大；把"装 + 清缓存"写进同一层后，镜像确实变小；能说出为什么 COPY 要放在装依赖之后；能说出多阶段构建解决了什么问题（一句话） 回滚方式：旧镜像 docker rmi my-hello:1.0 删掉即可；Dockerfile 改坏了就从 ~/lab/myimage 里的备份恢复
4-07	docker compose 起两个服务	07 册《Docker入门》· 关卡 4-07（该关即一节，标题同名）	能不看笔记写出一个两服务的 compose 文件；up -d / ps / logs / exec / down 五条命令能跑通；能用 curl 200 和 redis PONG 各验证一次；能说出 down 之后什么被删了、什么没被删 回滚方式：docker compose down；想连数据卷一起清就 down -v（练习环境随便清，生产环境别乱加 -v）
4-08	Compose 的网络与依赖顺序	07 册《Docker入门》· 关卡 4-08（该关即一节，标题同名）	能说出为什么代码里写 cache:6379 而不是 localhost:6379；能说出 depends_on 保证了什么、没保证什么；能用两种解法之一演示"依赖没就绪时不崩"；能用 docker network inspect 说出哪个网络里有哪几个容器 回滚方式：docker compose down；实验用的网络会跟着一起删
4-09	容器排障：日志 / 退出码 / 端口被占	07 册《Docker入门》· 关卡 4-09（该关即一节，标题同名）	能不看笔记走完三条排查链；能说出 137 和 143 各代表什么；能故意造一个"端口被占"并解决掉；能说出"容器 Up 但连不上"时先用 ss -tuln 判断占用，需查 PID/所有者时再用 sudo ss -tulnp 回滚方式：练习造的故障容器全部 docker rm -f 清掉；改动过的端口配置改回原样
4-10	阶段 4 · 学习总结练习：三个脚本容器化 + README	07 册《Docker入门》· 关卡 4-10（该关即一节，标题同名）	三个镜像都能 build 成功，且 docker history 里没有"又大又不该留"的层；改阈值用 -e 传进去能触发告警；compose 两个服务都是 Up；报告文件在宿主机目录里能找到；陌生人照 README 能跑通（写法见 16 册《一个合格的 README 长什么样》）；每一条都能说清"为什么这么做"

学到这 → 可以开始监控（阶段 5）。

阶段 5 • 监控（6 关） ｜ monitor-demo（Prometheus + Grafana + 阈值告警）

入场：4-10 完成 ｜ 毕业：监控链 + 阈值告警

编号	关卡名	在哪学（教材位置）	完成判定
5-01	监控地图：采集 / 存储查询 / 展示 先学：4-10	08 册《监控入门》· 关卡 5-01（该关即一节，标题同名）	能说出三个组件各自负责什么；能说出数据从哪来到哪去（一条链说下来）；能区分 Prometheus 默认的 1 分钟抓取/评估间隔与本实验显式设置的 15 秒；能说出本实验由 Prometheus 评估规则，Grafana 也能管理告警与通知；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）
5-02	用 compose 起 Prometheus + Grafana + node_exporter	08 册《监控入门》· 5-02	三个容器 Up；9090 / 9100 / 3000 可访问；Targets 的 node 为 UP；能先解释为何用 node-exporter:9100 而非 localhost；能 down / up -d 并复查。回滚：compose down（数据卷保留）
5-03	导入面板，看懂三条曲线	08 册《监控入门》· 关卡 5-03（该关即一节，标题同名）	数据源测试通过（Data source is working）；1860 面板导入成功且曲线在动；能说出三条曲线各自"什么样算不正常"；用 yes 制造波动并在曲线上看到；有一张截图存档 回滚方式：只停本终端作业 kill %1；若编号丢失，先用 pgrep -a yes 核对 PID、命令和所有者，再按明确 PID 处理
5-04	手写一条 PromQL：磁盘使用率	08 册《监控入门》· 关卡 5-04（该关即一节，标题同名）	能说出"指标名 + 标签"是什么关系；能默写出这条使用率公式并解释为什么用 100 减；会用 {mountpoint="/"} 过滤；能把这条查询放进 Grafana 面板 回滚方式：没有破坏性操作；实验面板删掉即可
5-05	配一条阈值告警规则	08 册《监控入门》· 5-05	规则文件正确、Prometheus 无报错；阈值改 1 后先预测再看到 PENDING→FIRING；能分清抓取间隔 15 秒、for: 1m、评估间隔；改回 85 并回到 INACTIVE
5-06	招牌关：用 Python 拉指标，超阈值报警 + crontab	08 册《监控入门》· 关卡 5-06（该关即一节，标题同名）	能画出返回结构（status → data → result → [0] → value[1]）并说出哪一步是数值；三种情况（正常 / 告警 / 连不上）都跑过，都有明确提示；日志文件里三种记录都有；crontab 里挂了任务，日志能看到新记录；能说清退出码 0 / 1 / 2 各自什么意思 回滚方式：先用 crontab -l 核对，再进入 crontab -e 只删除本次新增行；需要留底时先保存 crontab -l 输出。crontab -r 会删除该用户全部任务，不作为默认回滚；脚本和日志确认不再需要后再删

阶段 5 · 学习总结练习（见 08 册《监控入门》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。
学到这 → 可以开始 K8s（阶段 6）。

阶段 6 • Kubernetes（8 关） ｜ k8s-demo（含「Pod 起不来」踩坑记录）

入场：5-06 完成（"用 Python 拉指标"那关） ｜ 毕业：k8s-demo + 踩坑记录

编号	关卡名	在哪学（教材位置）	完成判定
6-01	打开 Killercoda，认识 kubectl 先学：5-06	09 册《K8s入门》· 关卡 6-01（该关即一节，标题同名）	能连上沙箱并跑通 kubectl get nodes；能说出 Pod / Deployment / Service 各解决什么问题；能说清应用流量为什么应优先通过 Service 访问 Pod，并能说明临时调试、进程诊断等直接访问 Pod 的例外场景；能说出 namespace 是干什么的；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） 回滚方式：沙箱是临时的，关掉浏览器就全清；不占本机资源，无回滚负担
6-02	部署第一个 Pod（Deployment + Service）	09 册《K8s入门》· 关卡 6-02（该关即一节，标题同名）	能不看教材写出一个 2 副本的 Deployment；能说出 selector 和 labels 为什么必须对上；能用 Service 名字从集群内访问到服务；能用 kubectl get all 说清现在集群里有什么 回滚方式：kubectl delete -f web.yaml 和 kubectl delete svc web-svc；沙箱关掉即清空
6-03	看：get / describe / logs	09 册《K8s入门》· 关卡 6-03（该关即一节，标题同名）	能用 -o wide 说出 Pod 被调度到哪台节点；能用 describe 的 Events 讲出一个 Pod 的启动时间线；能用 logs 看 Deployment 的日志；能说出"容器反复重启"时该加哪个参数（--previous） 回滚方式：只读操作，无风险
6-04	进容器：exec	09 册《K8s入门》· 关卡 6-04（该关即一节，标题同名）	能进 Pod 看文件并安全退出；能说出 -- 的作用；能不进容器直接执行一条命令；能说清"为什么在容器里改的东西 Pod 一重建就没了"，以及该用什么解决 回滚方式：删掉的 Pod 会自动重建（这就是下一关的内容）；文件改动随 Pod 一起消失，无残留
6-05	自愈：删掉 Pod 看它自动重建	09 册《K8s入门》· 关卡 6-05（该关即一节，标题同名）	删掉一个 Pod 并看到它被自动重建（截图/记录）；能说出"声明式"和"命令式"的区别（用删 Pod 这件事举例子）；能说出 spec 与 status 各是什么；能说出这一步在生产里有什么用 回滚方式：不用回滚 —— 实验本身就是删除，集群自己恢复
6-06	故意弄坏：镜像名写错，排查 ImagePullBackOff	09 册《K8s入门》· 6-06	能造出 ImagePullBackOff 并用 describe 找原因；能说出四类拉取失败关键词；先预测再核对故障稳定期的新旧 Pod 状态并解释副本数不等于 Pod 总数；修复后恢复 2/2 并留下排查记录。回滚：改回镜像并 apply
6-07	资源配额与调度：requests / limits	09 册《K8s入门》· 关卡 6-07（该关即一节，标题同名）	能说出 requests 和 limits 各自管什么（调度 vs 上限）；能造出一个 Pending 并用 describe 说出"为什么排不上"；能说出不写 limits 的后果（一个 Pod 拖垮整台节点）；能用 -o wide 说出 Pod 被调度到哪台节点；能说出 100m 是什么意思 回滚方式：kubectl delete pod hungry（Pending 的 Pod 也删得掉）；沙箱关掉即清空
6-08	阶段 6 · 学习总结练习：部署 → 弄坏 → 排查 → 留一份踩坑记录	09 册《K8s入门》· 关卡 6-08（该关即一节，标题同名）	三个故障都造出来过、都用 describe/logs 找到了根因；每个修复都有验证命令与输出；自愈实验有记录；有《Pod 起不来排查手册》，覆盖至少四种状态名（Pending / ImagePullBackOff / CrashLoopBackOff / Running 但访问不到）；能不看笔记讲 3 分钟"我是怎么查的"

学到这 → 可以开始综合项目（阶段 7）。

阶段 7 • 综合项目（8 关） ｜ server-health-check（王牌项目 v1）

入场：6-08 完成 ｜ 毕业：server-health-check（王牌项目 v1，长自阶段 3 的巡检 v0）

编号	关卡名	在哪学（教材位置）	完成判定
7-01	项目立项：巡检到底要检查什么 先学：6-08	10 册《综合项目》· 关卡 7-01（该关即一节，标题同名）	有一页检查清单（四项 + 门槛 + 异常话术）；能说出为什么每项都要有门槛；能说出退出码 0/1/2 各自代表什么；能说出这一版和 3-17 的 v0 差在哪（多了服务检查、门槛更明确）；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） 回滚方式：纯文档，无风险
7-02	v1：把四个检查写进脚本（subprocess + 阈值告警）	10 册《综合项目》· 7-02	脚本四项检查跑出结果；造一次告警并恢复；验证退出码 0/1/2；完成迁移检查：预测并验证 --disk-threshold 1 与 --skip-service 的输出/退出码；能解释函数封装与 subprocess 参数
7-03	输出报告与历史对比	10 册《综合项目》· 关卡 7-03（该关即一节，标题同名）	每次运行都生成一份带时间戳的报告文件；至少跑两次，能打印出"上次 vs 本次"的对比；挂上 crontab，等它自己跑出至少两份；能说出"基线"是什么、为什么没有基线就没法判断异常
7-04	容器化：把巡检脚本做成镜像	10 册《综合项目》· 关卡 7-04（该关即一节，标题同名）	镜像构建成功并能跑出巡检输出；报告通过挂载写入宿主机目录（容器删了报告还在）；能说清"为什么容器里 systemd 检查会失败"以及你选了哪种处理办法；能说出用 python:3.14-slim 当底座的好处
7-05	接到监控：把巡检指标暴露给 Prometheus	10 册《综合项目》· 关卡 7-05（该关即一节，标题同名）	/metrics 能返回至少三个自定义指标；Prometheus 的 Targets 里有 health-check 且 UP；Grafana 上能画出你自己的指标曲线；有一条自己的告警规则，且调低阈值能触发；能说出 Prometheus 的指标格式长什么样
7-06	README 与设计决策记录	10 册《综合项目》· 关卡 7-06（该关即一节，标题同名）	README 五段齐全，陌生人照它能跑通；有设计决策记录，至少覆盖四个决策；每条决策都写清"选了什么 / 还有什么 / 为什么不用"；仓库里没有密钥等敏感信息 回滚方式：文档改动可随时回退（git 里都有）
7-07	三分钟讲述稿与 10 分钟深挖	10 册《综合项目》· 关卡 7-07（该关即一节，标题同名）	三分钟讲述稿能不看稿讲完（录音为证）；能回答至少三个自检问题并指向项目里的具体文件或命令；讲述稿里至少说清一个"设计取舍"。回滚：无操作
7-08	阶段 7 · 学习总结练习：陌生人 clone 下来能跑	10 册《综合项目》· 关卡 7-08（该关即一节，标题同名）	在干净环境照 README 跑通（截图或记录）；三个跑法（本机 / 容器 / 指标）都验证过；README 里补上了"我卡住过的地方"；设计决策记录至少四条；三分钟讲述稿能不看稿讲完；仓库里无密钥、无大文件

学到这 → 可以开始 AI / GPU（阶段 8）。

阶段 8 • AI / GPU / 推理部署（10 关） ｜ ai-inference-notes（部署笔记 + 踩坑记录）

入场：7-08 完成 ｜ 毕业：ai-inference-notes（部署笔记 + 踩坑）

编号	关卡名	在哪学（教材位置）	完成判定
8-01	GPU 是谁：训练 vs 推理、显存 vs 内存 先学：7-08	11 册《AI与GPU入门》· 关卡 8-01（该关即一节，标题同名）	能用自己的话说清训练和推理的区别，并各举一个运维场景；能说出显存和内存是两回事，各用什么命令看；能用"博士 vs 小学生"讲清 GPU 为什么适合 AI（不掉进数学细节）；能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） 回滚方式：纯认知，无操作
8-02	用 Ollama 起一个本地模型并对话	11 册《AI与GPU入门》· 关卡 8-02（该关即一节，标题同名）	模型拉下来并能对话（截图或记录一句回答）；能说出模型文件放在哪、默认端口是多少、只听哪个地址；能用带 JSON 请求体的 POST 调一次生成接口并拿到 JSON；能按所选安装路线停服务、看到 Connection refused、再恢复。回滚按路线：A 原生用 ollama rm 与 systemctl；B Docker 用 docker exec ollama ollama rm、docker stop/rm；C 离线用实际二进制和记录的 PID
8-03	在 Colab 借一张免费 GPU，看懂 nvidia-smi	11 册《AI与GPU入门》· 关卡 8-03（该关即一节，标题同名）	能在 Colab（或 Kaggle）里拿到 GPU 并跑出 nvidia-smi；能逐列说出含义（至少：型号、驱动/CUDA、温度、功耗、显存、利用率）；能说出 GPU-Util 高和低各说明什么；能记录一次完整环境信息；知道抢不到卡时该怎么办 回滚方式：Colab 会话关掉即回收；不留本机痕迹
8-04	故意把显存撑爆：复现 CUDA out of memory	11 册《AI与GPU入门》· 关卡 8-04（该关即一节，标题同名）	按 11 册同名节的证据卡二选一：实机 CUDA OOM，或内存 OOM + “未实测 CUDA” 标注；两种都要能读出“现象 / 线索 / 建议” 、说出四类病因并提交排查记录。回滚：重启 GPU 会话或结束本机进程

编号	关卡名	在哪学（教材位置）	完成判定
8-05	跑一个最小训练	11 册《AI与GPU入门》· 关卡 8-05（该关即一节，标题同名）	两轮训练跑通，loss 明显下降；能说出 loss 下降说明什么；用两个不同 batch_size 各跑一次，说出显存占用的变化趋势；能说出"batch 大小 → 显存 → OOM"这条因果链 回滚方式：Colab 会话重启即清；下载的 MNIST 数据在 ./data，删掉即可
8-06	用 Python 调 DeepSeek API 做命令行助手	11 册《AI与GPU入门》· 关卡 8-06（该关即一节，标题同名）	脚本能带 --ask 与当前 DEEPSEEK_MODEL 提问并打印回答（或明确记录本轮未实测）；--local 能切到本地模型；能区分缺少 key、认证失败、余额不足和超时；key 与模型名都没有写死在代码里 回滚方式：删掉脚本与日志即可；key 在环境变量里，随时可在控制台吊销
8-07	用 vLLM 部署一个能访问的推理服务	11 册《AI与GPU入门》· 关卡 8-07（该关即一节，标题同名）	按 11 册 8-07 证据二选一：① vLLM 实机 /v1/models + 一次请求 + 日志；② Ollama 等价路径的端口监听 + /api/tags 200 + 一次生成请求 + 日志，并标注“未实测 CUDA / vLLM”。两条都要能说清回环边界和三类启动失败原因。回滚：停服务或关闭会话
8-08	把 GPU 指标接进监控面板	11 册《AI与GPU入门》· 关卡 8-08（该关即一节，标题同名）	按 11 册 8-08 证据二选一：实机把 dcgm-exporter 加入同一 Compose 项目并抓 dcgm-exporter:9400；无卡时启动 gpu-mock，抓 gpu-mock:9400/gpu.txt。两条都以 Prometheus GPU job 为 UP、Grafana 有显存/利用率两块面板和变化曲线为完成条件，降级证据必须标注“环境限制 / 未实测 GPU”。回滚：按 compose 服务名移除临时端点并恢复配置
8-09	读 DeepSeek-V3 技术报告，写 MoE / MLA 中文笔记	11 册《AI与GPU入门》· 关卡 8-09（该关即一节，标题同名）	能用大白话讲清 MoE / MLA；笔记各含至少一条原文证据（章节号或标题 + 图表编号 + 证明了自己的哪个理解）；至少一条真实疑问；AI 助手只用于解释术语，不作为事实来源。回滚：纯阅读，无风险
8-10	画一张全景图，讲 10 分钟	11 册《AI与GPU入门》· 关卡 8-10（该关即一节，标题同名）	有一张手画的全景图（六层齐全）；每层能说出一个"会出什么问题"和一个"看什么指标"；能对着图不卡壳讲满 10 分钟（录音为证）；至少能回溯到三个自己练过的关卡（点名编号） 回滚方式：无操作

阶段 8 · 学习总结练习（见 11 册《AI 与 GPU 入门》 书末同名小节：业务场景、综合要求、验收与产出都在那一节）——本表只记关卡号与判定，正文不重复。

总索引 • 要查什么 → 翻哪册

你要找	去哪册
Linux 命令、权限、进程、服务、网络、Shell 脚本	01 Linux 入门（练：02 基础篇 / 03 进阶篇）
Python 语法、模块、文件、异常、自动化	04 Python 入门（练：05 基础篇 / 06 进阶篇）
内置函数、字符串方法、文件与 JSON 写法	12 内置函数手册
报错里的英文单词看不懂	13 报错英语单词手册
日志读不懂、报错不会定位	14 日志报错解读手册
变量 / 函数怎么起名	15 命名手册
代码提交、推送到 GitHub	16 Git 与 GitHub 速查
装第三方库（pip / venv / requirements）	04 Python 入门 · 附录 E
我到底学会了没有	17 关卡总表（逐条核对）

附录 A • 检验点（按关卡归位）

检验点按关卡归位（与各册章末的作业 / 题目配套）。

01 • Linux 入门（教科书）

第 0 章 • 开篇：终端与命令 → 关卡 1-01、1-02

Linux 是什么、为什么运维要学它
能打开终端并看懂提示符
会使用"命令 + 选项 + 参数"三件套
会用 --help 自救

第 1 章 • 目录树与路径 → 关卡 1-03、1-04

理解"一切皆文件"和目录树
说出三大重要目录的用途
会区分绝对路径与相对路径
会用 cd 在树里移动

第 2 章 • 文件与目录操作 → 关卡 1-05、1-06、1-07

会建删件移动录改名
理解删除的不可逆
会用 mkdir -p 连级创建

第 3 章 • 查看与搜索 → 关卡 1-08、1-09、1-10

能跟读并说出 26.04 版 ls -lh / du -sh 的完整数字含义，不再靠猜。

第 4 章 • 管道与重定向 → 关卡 1-11、1-12、1-13

理解管道 | 的流水线思想
区分 >（覆盖）与 >>（追加）
会 sort / uniq 配合管道
awk 入门：按列取数据

第 5 章 • 用户与权限 → 关卡 1-14、1-15、1-16

能用符号法设置权限，并主动说"我只给需要的人"。

第 6 章 • 进程管理 → 关卡 1-17、1-18、1-19

看懂 ps aux 各列
会找进程（pgrep / ps+grep）
理解"优雅杀 vs 强杀"
会前后台切换与 nohup
会用 fuser 定位占端口的进程

第 7 章 • 磁盘与文件系统 → 关卡 1-20、1-21、1-22

会用 df 看整体、du 看明细
理解"挂载"
会造测试大文件（fallocate / dd）
看懂磁盘满的排查套路。

第 8 章 • 网络命令 → 关卡 1-23、1-24、1-25、1-26

会查 IP、网关（ip / ip route）
会用 ping、ss、nc、curl 做基础排查
会 ssh / scp 远程操作
掌握"主机→端口→进程→日志"四层排查法

第 9 章 • apt 包管理 → 关卡 1-27

理解包管理器为什么存在
会用 apt 装/卸/搜软件
看懂常见的锁文件报错

第 10 章 • systemctl 与服务 → 关卡 1-28、1-29、1-30

理解 systemd 和"服务"
会用 systemctl 管理服务全流程
会用 journalctl 看服务日志

第 11 章 • 环境变量 → 关卡 1-31

理解环境变量是"全局公告栏"
明白 PATH 决定"命令找不找得到"
会 export、改 rc 文件、source
会 alias 起外号

第 12 章 • 归档与压缩 → 关卡 1-32

理解"归档"和"压缩"是两件事
会用 tar 打包（含 tar.gz）
会 gzip / zip
会用 diff 验证解包无误

第 13 章 • Shell 脚本基础 → 关卡 1-33、1-34、1-35、2-05、2-08、2-09

理解"脚本 = 把敲过的命令装进文件"
会写带变量、if、for 的脚本
会用 crontab 定时执行
知道脚本的标准写法

02 • Linux 练习册 • 基础篇

自测项 → 判定题挂在 01 册对应阶段各关

能独立完成 阶段 1 | 文件系统操作：导航、创建、复制、移动、删除
能独立完成 阶段 2 | 查看与管道：cat/less/head/tail/grep/重定向
能独立完成 阶段 3 | 权限与用户：chmod/chown/sudo/用户管理
能独立完成 阶段 4 | 进程与系统：ps/top/kill/服务/磁盘/包管理
能独立完成 阶段 5 | 查找与归档：find/grep -r/sort/uniq/tar

03 • Linux 练习册 • 进阶篇

自测项 → 挂在 03 册 5 个场景各关

能独立完成 场景 1 | 本地推理服务起不来（日志排查）
场景 2 | 磁盘满应急（☆ 选做）：能独立定位大文件/大目录并给出清理方案（含"先看再删"的检查顺序）。
能独立完成 场景 3 | 服务连不上（网络链路排查）
能独立完成 场景 4 | 服务与进程运维（含 GPU 进程定位）
能独立完成 场景 5 | 脚本收尾（衔接 Shell 脚本任务）

04 • Python 入门（教科书）

第 0 章 • Python 与 AI 运维的关系 → 关卡 3-01 ~ 3-10

Python 是什么、为什么 AI 运维要用它
会装好环境并跑出第一行代码
知道 Python 和 Shell 怎么分工

第 1 章 • 起步：print、变量、类型 → 关卡 3-01 ~ 3-10

能写"自我介绍卡"程序——程序打印出名字、年龄，还会算两位数倒序。这是所有 Python 代码的地基

第 2 章 • 控制流：if、for、while → 关卡 3-01 ~ 3-10

会写成绩分段程序、九九乘法表、猜数字游戏——程序从此会"自己拿主意"和"重复干活"

第 3 章 • 数据结构：list、dict、tuple、set → 关卡 3-01 ~ 3-10

会写通讯录小程序（能存、能查、能删）。数据结构是"数据处理"的核心地基——后面处理日志、JSON、API 全靠它们

第 4 章 • 函数：def、参数、返回值 → 关卡 3-01 ~ 3-10

写自己的函数（问候、计算、判断）+ 一个"计算器"函数集合——代码从此可以打包复用

第 5 章 • 模块与导入：import、os、time、random → 关卡 3-01、3-02

写"我的工作目录报告"——列出当前目录所有文件和修改时间。你开始调用 Python 自带的大工具箱（标准库）
★ 学完本章 = 达成检查点①：能独立完成第 5 章全部章末作业和自测清单（逐项运行、逐项核对）

第 6 章 • 文件操作：open/with、json → 关卡 3-03、3-04

把通讯录存成 JSON 文件再读回来——程序从此能记住数据（关掉再开不丢失）

第 7 章 • 异常处理：try/except/finally → 关卡 3-05

★ 给第 6 章的 JSON 程序加上"文件不存在时友好提示"——你的脚本从此不会一碰错误就崩溃。学完本章 = 达成检查点②：能独立写一个"文件不存在时给人话提示"的脚本（运行一次并留下记录）

第 8 章 • 网络请求：requests → 关卡 3-06、3-07

写"天气查询器"雏形——程序联网拿数据。这是脚本 2 的核心能力，也是调 API 的第一步

第 9 章 • 命令行参数：argparse → 关卡 3-08

脚本支持 python3 weather.py 杭州 这种命令行调用——同一脚本应对不同情况，从"玩具"变"工具"

第 10 章 • 日志：logging → 关卡 3-09

给脚本加日志记录——每次操作留痕，出问题能排查。日志就是运维的第一现场（你读过 Linux 日志，现在自己生产日志）

第 11 章 • 进程调用：subprocess → 关卡 3-10

用 Python 调 df -h、free -h 并处理结果——Linux 三连和 Python 三连在这里合流。本章是通往王牌项目 server-health-check 的接口

05 • Python 练习册 • 基础篇

自测项 → 判定题挂在 04 册对应阶段各关

能独立完成 阶段 1 | 变量与类型
能独立完成 阶段 2 | 控制流
能独立完成 阶段 3 | 数据结构
能独立完成 阶段 4 | 函数与模块
能独立完成 阶段 5 | 文件与异常
能独立完成 阶段 6 | 模块组合

06 • Python 练习册 • 进阶篇

自测项 → 挂在 06 册 5 个场景各关

能独立完成 场景 1 | 批量文件重命名工具
能独立完成 场景 2 | 天气查询器
能独立完成 场景 3 | 本机巡检工具雏形 ★（王牌项目版本 0 · 教学简化版）
能独立完成 场景 4 | Nginx 日志分析器
能独立完成 场景 5 ☆ | paramiko 远程巡检雏形（选做）

12 • Python 内置函数手册

自测项 → 关卡 字典，不占关卡

能按分组快速查到内置函数的用法（输入输出 / 类型 / 统计 / 序列 / 检查）
会查文件与 JSON 的读写写法（open / with / json.load / json.dump）
掌握 11 组、共 15 个常用字符串方法并会查用（split / join / strip / lstrip / rstrip / replace / upper / lower / find / startswith / endswith / count / rsplit / splitlines / isdigit）
能看懂「替代写法对比」，知道哪种更推荐

13 • 报错英语单词手册

自测项 → 关卡 字典，不占关卡

能说出报错英语的构造规律（倒着装回中文语序）
A~F 六组高频词过一遍，看到能认出意思
能做对 6 道「看报错猜意思」自测题
看到陌生报错，知道先找哪个关键词

14 • 日志报错解读手册

自测项 → 关卡 字典，不占关卡

能读懂日志结构、Python logging 标准级别 DEBUG/INFO/WARNING/ERROR/CRITICAL（部分工具可显示 WARN 简写）
A 组 Linux 报错前 10 条能条件反射说出含义
B 组 Python 报错看到 Traceback 会定位到最后一行
会用「排错心法」处理没见过的报错

15 • 命名手册

自测项 → 关卡 字典，不占关卡

起名三原则能背（见名知意 / 下划线 / is_ 判断）
会用「起名四步法」给新变量命名
知道哪些是 Python 内置名，不会撞名（list / sum / max…）
记住 pwd 有歧义，密码写 password

16 • Git 与 GitHub 速查

自测项 → 关卡 字典，不占关卡

配好 SSH 钥匙，ssh -T git@github.com 能通过 ★
能独立完成「改 → add → commit → push」循环
会写 .gitignore（.venv / __pycache__ / *.log / 密钥）
提交信息写得清楚（别人看得懂你做了什么）
GitHub 上有自己的仓库，且提交记录 ≥ 10 次 ★

附录 B • 外部资源清单（在哪学、只做哪一部分）

全部外部资源都在这里（都免费或有免费额度）；按"只做这一部分"照做即可，不用自己到处试。要注册、要额度、可能失效的，都写了备注和备选。

资源	用在哪几关	只做这一部分	备选
Ubuntu 系统本身 WSL2（推荐）/ VMware	0-03	只做：Windows 上用 `wsl --install -d Ubuntu`（推荐），或 VMware + 清华镜像的 26.04 ISO——装法见 01 册 · 附录 E	平板 Termux 路径：自行按 Termux 官方文档或搜索安装教程
Ubuntu 镜像下载 mirrors.tuna.tsinghua.edu.cn/ubuntu-releases/	0-03（VMware 路线）	只下 26.04 的 Desktop ISO 一个文件；走 WSL 路线不用下	阿里云镜像站 / 官网 ubuntu.com
Killercoda killercoda.com/playgrounds	6-01 ~ 6-08	只选 **Kubernetes** 那个 playground，在右侧终端里敲 `kubectl`；用完关掉页面即可	Play with Kubernetes（docker labs）；本机 minikube（不推荐）
Google Colab colab.research.google.com	8-03 / 8-04 / 8-05	只做三步：新建笔记本 →「代码执行程序 → 更改运行时类型 → GPU」→ 跑 `!nvidia-smi`；抢不到卡就过两天再来（不是你的操作问题）；**国内网络打不开 Colab 就用备选 Kaggle（步骤同上）**	**Kaggle Notebooks**（需账号，GPU 额度更宽松）
Kaggle kaggle.com	8-03 备选	New Notebook → Settings → Accelerator 选 GPU，其余步骤同 Colab	Colab
Ollama ollama.com	8-02 / 8-06	只做：装 → `ollama pull qwen2.5:1.5b` → 原生 `ollama run qwen2.5:1.5b` 或 Docker `docker exec -it ollama ollama run qwen2.5:1.5b`；再用带 JSON 请求体的 POST 调一次：`curl -sS http://localhost:11434/api/generate -H 'Content-Type: application/json' -d '{"model":"qwen2.5:1.5b","prompt":"只回复OK","stream":false}'`	llama.cpp；或用 Docker 版镜像
DeepSeek API platform.deepseek.com	8-06	只做：建一个 API Key 放进环境变量 → 用脚本调一次；**没 key 就用 `--local` 走本地 Ollama，一样能过关**	本地 Ollama（脚本已内置）；其它同类 API 按需替换 URL
Docker Hub hub.docker.com	4-01 / 4-02 / 5-02	只拉这几个官方镜像：`hello-world`、`nginx:alpine`、`redis:7-alpine`、`prom/prometheus`、`prom/node-exporter`、`grafana/grafana`	国内镜像加速器（拉不动时配置）
Prometheus / Grafana	5-01 ~ 5-06	只用默认端口：Prometheus 9090、Grafana 3000、node-exporter 9100；面板只导入 `1860` 一个	Grafana Play（在线演示站）
GitHub github.com	0-01 / 0-02；各产出仓库	只做：建仓库 → `git push` → 打开仓库页确认文件在；需要在线看教材时用 Pages	Gitee（国内备选）
Open-Meteo api.open-meteo.com	3-13（天气查询器）	只用一个地址：`/v1/forecast?latitude=..&longitude=..&current_weather=true`；不要研究它的其它参数	wttr.in 等免 key 天气接口
Ubuntu 官方源 archive.ubuntu.com	1-27（apt）	只用 `apt update / install / search` 三条；换源只在拉不动时才做	国内镜像源（清华 / 阿里）
PyPI / pip 源 pypi.org	04 册附录 E / 06 · 11 册 pip	只用 `pip install 包名` 一条；**国内拉不动就换清华镜像源**（04 册附录 E 有写法）	清华 PyPI 镜像
HuggingFace huggingface.co	8-07（vLLM 拉模型）	只拉一个模型：`Qwen/Qwen2.5-1.5B-Instruct`	hf-mirror.com（国内镜像）
NVIDIA NGC（nvcr.io） nvcr.io	8-08（dcgm-exporter 镜像）	只拉一个镜像：`nvcr.io/nvidia/k8s/dcgm-exporter`；**没有 N 卡就按 8-08「环境兜底」走降级演练**	无（该镜像只此来源；无卡走降级）